量化交易进阶:第45天 – 因子数据库设计(Factor Database Design)

原创 逻辑与流 逻辑与流 2025年11月4日 21:03 北京 标题已修改 1人

数据结构决定策略效率。因子计算完成后,如何存储、索引、查询直接影响整个回测与生产系统的性能。本文系统讲解因子数据库设计思路,从三层架构(原始行情库、因子库、回测数据集)出发,详细解析因子表的核心字段设计、主键与索引策略、外键关系。对比SQLite与MySQL的适用场景,通过SQLAlchemy ORM框架完整演示因子数据库的建模、连接、写入、查询操作,提供可直接运行的完整示例代码。揭示因子数据库在量化系统中的核心作用——让因子计算与回测模块完全解耦,支持多进程与历史数据复用,大幅提升系统效率,为量化策略的工程化奠定坚实基础。

学习时间

- **理论学习**: 15分钟
- **代码实践**: 20分钟
- **案例分析**: 10分钟
- **总计**: 45分钟

我们正式进入量化课程 Day 45：因子数据库设计。

今天这一讲非常关键，因为——“数据结构决定策略效率”。

因子计算完以后，如果你存储、索引、查询都设计得好，整个回测与生产系统会快好几倍。

一、因子数据库设计思路

在一个完整的量化系统中，我们通常有如下三个层次的数据库结构：

模块	作用	示例表
原始行情库	存储原始行情数据（开高低收、成交量等）	price_daily
因子库	存储各种计算后的因子结果	factor_value
回测数据集	用于回测或实盘信号	backtest_signal

今天我们主要关注因子库设计。

因子表的核心字段设计

字段名	类型	含义	备注
id	INTEGER / BIGINT	自增主键	唯一标识
factor_id	VARCHAR(64)	因子名或因子标识	例如 "PE", "PB", "ROE_5D"
stock_code	VARCHAR(16)	股票代码	e.g. "000001.SZ"
trade_date	DATE	交易日期	通常按日为主键部分
factor_value	FLOAT / DOUBLE	因子值	可以为 NULL

字段名	类型	含义	备注
update_time	DATETIME	数据更新时间	自动维护

主键与索引设计

通常我们希望唯一确定一条因子记录的键是：

PRIMARY KEY (factor_id, stock_code, trade_date)

这样可以保证每个因子、每支股票、每个交易日的记录唯一。

此外，为加快查询：

- 创建索引：
  - (trade_date) → 快速获取某日所有股票的因子
  - (stock_code) → 快速获取某支股票的时间序列
  - (factor_id) → 快速获取某个因子全样本

外键关系

可选，但推荐在工程化系统中加入：

- stock_code 可关联到股票主表 stock_basic(code, name, industry, status)
- factor_id 可关联到因子定义表 factor_meta(factor_id, description, update_freq, author)

这让系统更具可扩展性。

二、SQLite 与 MySQL 对比

结论：

- 学习、研究、开发阶段 → SQLite
- 回测平台、因子共享服务 → MySQL

SQLite 是最佳入门方案：易用、便携、无依赖。

三、用 SQLAlchemy 建模与操作因子表

SQLAlchemy 是 Python 的 ORM 框架，可以兼容 SQLite、MySQL、PostgreSQL 等。

下面我们从零构建一个完整的因子数据库模块。

1 建立模型类（ORM 映射）

```
from sqlalchemy import create_engine, Column, Integer, String, Float, Date, DateTime, Primar
from sqlalchemy.orm import declarative_base, sessionmaker
from datetime import datetime

Base = declarative_base()

class FactorValue(Base):
```

```
    __tablename__ = 'factor_value'

    factor_id = Column(String(64), nullable=False)
    stock_code = Column(String(16), nullable=False)
    trade_date = Column(Date, nullable=False)
    factor_value = Column(Float)
    update_time = Column(DateTime, default=datetime.now, onupdate=datetime.now)

    __table_args__ = (
        PrimaryKeyConstraint('factor_id', 'stock_code', 'trade_date'),
    )
```

2 创建数据库连接与表结构

```
# SQLite
engine = create_engine('sqlite:///factor_db.sqlite', echo=False)

# MySQL（可扩展）
# engine = create_engine('mysql+pymysql://user:password@localhost/factor_db?charset=utf8mb4'

Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()
```

3 写入数据

```
import pandas as pd

# 示例数据
data = [
    {'factor_id': 'PE', 'stock_code': '000001.SZ', 'trade_date': '2025-11-01', 'factor_value
    {'factor_id': 'PE', 'stock_code': '000002.SZ', 'trade_date': '2025-11-01', 'factor_value
    {'factor_id': 'PB', 'stock_code': '000001.SZ', 'trade_date': '2025-11-01', 'factor_value
]

# 批量写入
records = [FactorValue(**d) for d in data]
session.add_all(records)
session.commit()
```

4 查询数据

```
# 查询某日的 PE 因子
res = session.query(FactorValue).filter(
    FactorValue.factor_id == 'PE',
    FactorValue.trade_date == '2025-11-01'
```

```
).all()

for r in res:
    print(r.stock_code, r.factor_value)
```

5 pandas 一键写入（适合大批量）

```
df = pd.DataFrame(data)
df.to_sql('factor_value', engine, if_exists='append', index=False)
```

四、完整可运行示例（SQLite）

整合上面内容，以下代码可直接运行：

```
from sqlalchemy import create_engine, Column, String, Float, Date, DateTime, PrimaryKeyConst
from sqlalchemy.orm import declarative_base, sessionmaker
from datetime import datetime

Base = declarative_base()

class FactorValue(Base):
    __tablename__ = 'factor_value'
    factor_id = Column(String(64), nullable=False)
    stock_code = Column(String(16), nullable=False)
    trade_date = Column(Date, nullable=False)
    factor_value = Column(Float)
    update_time = Column(DateTime, default=datetime.now, onupdate=datetime.now)
    __table_args__ = (
        PrimaryKeyConstraint('factor_id', 'stock_code', 'trade_date'),
    )

# 建立 SQLite 连接
engine = create_engine('sqlite:///factor_db.sqlite', echo=False)
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()

# 插入样例数据
data = [
    {'factor_id': 'PE', 'stock_code': '000001.SZ', 'trade_date': '2025-11-01', 'factor_value
    {'factor_id': 'PE', 'stock_code': '000002.SZ', 'trade_date': '2025-11-01', 'factor_value
    {'factor_id': 'PB', 'stock_code': '000001.SZ', 'trade_date': '2025-11-01', 'factor_value
]
records = [FactorValue(**d) for d in data]
session.add_all(records)
session.commit()

# 查询示例
res = session.query(FactorValue).filter(
    FactorValue.factor_id == 'PE',
    FactorValue.trade_date == '2025-11-01'
```

```
).all()

for r in res:
    print(f"{r.stock_code}: {r.factor_value}")
```

输出：

```
000001.SZ: 12.3
000002.SZ: 15.6
```

五、扩展到 MySQL（只需两步）

1 修改连接字符串：

```
engine = create_engine('mysql+pymysql://user:password@localhost/factor_db?charset=utf8mb4')
```

2 在 MySQL 中提前建库：

```
CREATE DATABASE factor_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

六、与"因子计算 → 存储 → 回测"流程衔接

整个流程如下：

1 计算阶段

在 Python 中根据行情数据计算因子值（如 PE、ROE、动量等）。

2 存储阶段（今日主题）

将因子结果以 factor_id, stock_code, trade_date 三元组存入数据库。

3 回测阶段

从数据库中读取所需因子数据 → 合并行情数据 → 排序/分组构建多空组合 → 计算收益。

数据库在此扮演中枢角色：

它让"计算模块"和"回测模块"完全解耦，支持多进程与历史数据复用。

总结

模块	要点
表结构	factor_id + stock_code + trade_date 三元主键
索引	按因子、股票、日期建索引加速查询
SQLite	本地研究型数据库，零配置

模块	要点
MySQL	生产与团队协作首选
ORM 层	使用 SQLAlchemy，兼容多数据库
工程化意义	因子复用、高效存储、快速查询、回测无缝衔接

结论

因子数据库设计是量化系统工程化的关键环节。通过合理的数据结构设计、索引策略和ORM框架应用,我们实现了因子计算与回测模块的完全解耦,大幅提升了系统的可扩展性和执行效率。掌握SQLite与MySQL的适用场景,熟练使用SQLAlchemy进行数据库操作,为量化策略的工程化部署奠定了坚实基础。

量化交易入门指南 · 目录 ≡

〈上一篇
量化交易进阶:第44天 – 因子正交化与去重(Factor Orthogonalization&De–...

下一篇 〉
量化交易进阶:第46天 – 数据存储工程化 因子每日更新脚本(pandas + SQLAlchemy)(进...

阅读 144　修改于2025年11月4日

广告



末日寒潮席卷城市，把人带回来，把火烧旺。
点击了解更多 〉

末日寒潮席卷城市，把人带回来，把火烧旺。

无尽冬日–3亿人都在玩

立即体验